

Estruturas de Dados e Análise de Algoritmos

A3 - Estudo de Caso Aplicado

Caso: Otimização de Tempos de Carregamento no GTA Online.

Integrantes:

Elizabeth Stéphany Guimarães Miranda. 123220604

Ana Luiza Mattos de Carvalho. 124114111

Camila Ferreira dos Santos 124117665

Miguel Pedro Pinheiro. 12315515

1. Escolha e Apresentação do Caso

a) Empresa e Produto

A Rockstar Games é uma das maiores desenvolvedoras de jogos do mundo. O produto analisado é o GTA Online, o modo multijogador massivo do Grand Theft Auto V, lançado originalmente em 2013 e que se mantém como um dos produtos mais lucrativos da indústria do entretenimento.

b) Contextualização do Problema

Durante anos, os jogadores de GTA Online enfrentaram telas de carregamento excessivamente longas ao iniciar o modo multiplayer, variando facilmente de 5 a 15 minutos, mesmo em computadores topo de linha. A comunidade aceitava isso de forma anedótica como "carregamento infinito" ou "problema dos servidores da Rockstar".

Em 2021, um desenvolvedor e jogador sob o pseudônimo t0st decidiu investigar o que o jogo estava fazendo durante esse tempo, já que o uso de CPU disparava em um único núcleo, enquanto o uso de disco e rede ficava praticamente zerado.

c) Impacto no Negócio e no Sistema

- Retenção de Usuários: Barreiras de entrada longas (esperar 10 minutos para jogar) desmotivam o jogador, reduzindo o *Daily Active Users* (DAU).
- Experiência do Usuário (UX): Frustração generalizada da comunidade e saturação de reclamações em fóruns de suporte.
- Custo Computacional Inútil: Desperdício de processamento de hardware no lado do cliente com loops redundantes.

2. Raio-X Técnico do Caso

a) Arquitetura e Stack Tecnológica

- Linguagem: C++ (padrão da engine proprietária RAGE - *Rockstar Advanced Game Engine*).
- Formato de Dados: JSON (um arquivo de texto de aproximadamente 10 MB contendo uma lista de 63.000 itens/itens de customização compráveis no jogo).
- Infraestrutura/Processamento: Execução local (Client-side) durante a inicialização, rodando estritamente em uma única *thread* (single-core).

b) Problema Computacional Envolvido

Ao analisar o binário do jogo com ferramentas de *reversing* (como o x64dbg), t0st descobriu duas falhas gravíssimas de algoritmo ao processar o arquivo JSON de 10 MB:

1. Parser de JSON Ineficiente: O jogo utilizava uma função de leitura de strings que recalculava o tamanho da string (`strlen`) a cada caractere lido do arquivo.
2. Deduplicação por Busca Linear: Para cada um dos 63.000 itens lidos do JSON, o código realizava uma verificação para garantir que o item já não estava cadastrado em um array (lista). O algoritmo realizava um `array.find()` que percorria a lista sequencialmente do início ao fim.

Por que isso era difícil? À medida que o tamanho do arquivo (n) crescia, o tempo de execução escalava de forma quadrática. Para 63.000 itens, o pior caso do algoritmo executava aproximadamente $63.000^2 \approx 3.9 \times 10^9$ (quase 4 bilhões) de operações de comparação, travando o núcleo da CPU em processamento inútil.

c) Classificação Conceitual do Problema

- Domínio: Busca e Processamento de Dados.
- Complexidade do Algorítmico Original: $O(n^2)$ (Complexidade Quadrática).
- Classificação de Complexidade: O problema em si reside na classe **P** (tempo polinomial), mas foi implementado de forma ingênua, tornando-se inviável para grandes volumes de dados.

d) Estratégia Utilizada pela Empresa (Originalmente)

A Rockstar utilizou uma estrutura de Array/Vetor Dinâmico sequencial. Toda vez que um novo item era lido, o código varria o array inteiro para verificar duplicatas antes de inseri-lo no final.

3. Proposta Alternativa do Grupo

A proposta do grupo foca na alteração da estrutura de dados utilizada para a verificação de duplicidade e armazenamento temporário durante o carregamento.

Substituição de Array por Tabela Hash (HashMap)

Em vez de injetar os itens em um vetor dinâmico e realizar uma busca sequencial $O(n)$ a cada iteração, a estratégia consiste em:

1. Instanciar uma tabela hash (HashMap ou `std::unordered_set` em C++).
2. Para cada item lido do JSON, disparar uma função hash sobre o identificador único do item.
3. Verificar a existência do item diretamente na tabela hash.
4. Se não existir, o item é inserido tanto na tabela hash quanto na lista final de itens do jogo.

Justificativa Técnica e Desempenho

- A busca e a inserção em um HashMap bem dimensionado possuem complexidade de tempo médio de $O(1)$ (Tempo Constante).
- Ao alterar a estratégia, a complexidade total do algoritmo cai de $O(n^2)$ para $O(n)$ (Tempo Linear).
- Em termos práticos: em vez de fazer 4 bilhões de comparações, o sistema faz apenas cerca de 63.000 operações.

Vantagens e Limitações

- Vantagem Principal: Ganho brutal de desempenho. O tempo de processamento cai de minutos para milissegundos.
- Limitação/Trade-off: Pequeno incremento no consumo de memória RAM para armazenar os ponteiros e índices da tabela hash (completamente desprezível para computadores modernos rodando um jogo desse porte).

4. Análise Crítica

Trade-offs: Solução Ótima vs. Solução Viável

O caso ilustra perfeitamente a frase do professor: *"Nem sempre a melhor solução teórica é a melhor solução prática"*. Quando o GTA Online foi programado em 2013, a lista de itens (n) devia ser muito pequena, tornando o impacto do algoritmo $O(n^2)$ imperceptível nos testes internos. O erro da empresa foi negligenciar o impacto da escalabilidade a longo prazo: com o acúmulo de DLCs ao longo de 8 anos, a base de dados cresceu e o algoritmo se degradou terrivelmente.

Impacto Real do Conserto

O desenvolvedor t0st provou que sua alteração reduziu o tempo de carregamento do jogo em 70% (de mais de 6 minutos para apenas 1 minuto e 50 segundos). A Rockstar Games reconheceu publicamente o erro do código, agradeceu formalmente ao jogador, adotou oficialmente a correção via patch de atualização e o premiou com US\$ 10.000 através do seu programa de *Bug Bounty*.

5. Referências

1. **t0st (Blog Oficial):** *"How I cut GTA Online loading times by 70%"*, 2021.

2. **Rockstar Games Support:** Notas de Atualização oficiais do Patch 1.53 do GTA V/GTA Online.
3. **Cormen, T. H. et al.** *Algoritmos: Teoria e Prática*. Rio de Janeiro: Elsevier (Apoio teórico sobre tabelas Hash e Notação Big-O).